Bloque Práctico: Comparativa en Python

Clase 15 — K-Means vs. Clustering Jerárquico vs. DBSCAN

Duración: 40 minutos — scikit-learn y scipy

En este bloque los estudiantes comparan K-Means, Clustering Jerárquico y DBSCAN sobre un dataset geométricamente complejo (dos lunas entrelazadas), donde K-Means falla por su supuesto de forma esférica. El objetivo es que la comparación visual haga evidente por qué la elección del algoritmo depende de la geometría de los datos, no solo del número de grupos esperado.

Paso 1 — Generación y estandarización de datos

Se genera un dataset sintético de dos lunas entrelazadas con make_moons, una geometría no convexa que K-Means no puede separar correctamente. La estandarización con StandardScaler es obligatoria: tanto el clustering jerárquico como DBSCAN se basan en distancias euclídeas, y una variable con escala mayor dominaría el cálculo si no se normaliza.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler

# Generamos un dataset con dos lunas entrelazadas
X, y_real = make_moons(n_samples=300, noise=0.08, random_state=42)

# Estandarizamos las variables (indispensable para métricas de distancia)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Paso 2 — Dendrograma con SciPy

scipy.cluster.hierarchy.linkage calcula la matriz de enlace (el historial completo de fusiones). dendrogram la grafica como árbol invertido. La línea roja en y=3.5 muestra un corte hipotético: todo punto donde una fusión ocurre por encima de esa altura queda en ramas separadas, definiendo los clusters finales sin haber fijado K de antemano.

```
from scipy.cluster.hierarchy import dendrogram, linkage, cut_tree

# Calculamos la matriz de enlace usando Complete Linkage
matriz_enlace = linkage(X_scaled, method='complete')

# Graficamos el dendrograma
plt.figure(figsize=(10, 5))
dendrogram(matriz_enlace, color_threshold=3.5, above_threshold_color='black')
plt.axhline(y=3.5, color='r', linestyle='--', label='Corte a altura 3.5')
plt.title('Dendrograma - Clustering Jerárquico Aglomerativo')
plt.xlabel('Índice de la observación')
plt.ylabel('Distancia / Disimilitud')
plt.legend()
plt.show()
```

Paso 3 — Entrenamiento y comparación de los tres modelos

Se entrenan los tres algoritmos sobre los mismos datos estandarizados y se grafican lado a lado. K-Means, al forzar fronteras lineales entre centroides, corta cada luna por la mitad. El jerárquico con complete linkage mejora algo el

resultado pero sigue sesgado hacia clusters compactos. DBSCAN, al basarse en densidad y no en centroides, es el único que recupera la forma real de ambas lunas.

Nota: en el código original los ejes del subplot se indexaban como axes (sin corchetes) y axes[14], lo cual produce un error porque plt.subplots(1, 3) devuelve un arreglo de 3 ejes (índices 0, 1 y 2). Aquí está corregido a axes[0], axes[1], axes[2].

```
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN

# 1. K-Means
kmeans = KMeans(n_clusters=2, n_init=20, random_state=42)
labels_kmeans = kmeans.fit_predict(X_scaled)

# 2. Clustering Jerárquico Aglomerativo
hierarchical = AgglomerativeClustering(n_clusters=2, linkage='complete')
labels_hierarchical = hierarchical.fit_predict(X_scaled)

# 3. DBSCAN
dbscan = DBSCAN(eps=0.3, min_samples=5)
labels_dbscan = dbscan.fit_predict(X_scaled)

# Graficamos los 3 resultados lado a lado
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))

axes[0].scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels_kmeans, cmap='viridis')
axes[0].set_title('K-Means (corta formas complejas)')

axes[1].scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels_hierarchical, cmap='viridis')
axes[1].set_title('Jerárquico - Complete Linkage')

axes[2].scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels_dbscan, cmap='viridis')
axes[2].set_title('DBSCAN (capta la geometría real)')

plt.tight_layout()
plt.show()
```

Ejercicios propuestos

Los tres ejercicios siguientes están resueltos en notebooks separados (uno por ejercicio). Aquí se resume la consigna y el criterio de evaluación de cada uno.

Ejercicio 1 — Corte del dendrograma y asignación de etiquetas

- Dataset: USArrests (indicadores de criminalidad por estado, 4 variables numéricas).
- Se aplica AgglomerativeClustering con linkage='complete' sobre las variables estandarizadas.
- Se usa cut_tree para extraer etiquetas con 3 y 5 clusters.
- Se verifica con pd.crosstab que los 5 subgrupos se anidan dentro de los 3 grupos principales — esa anidación es la propiedad definitoria de un clustering jerárquico.

Ejercicio 2 — Comparativa de criterios de enlace

- Se entrenan tres modelos variando linkage entre 'complete', 'single' y 'average'.
- Se grafican los tres dendrogramas resultantes sobre el mismo dataset.
- Análisis esperado: 'single' es el que genera el efecto de cadenas (trailing clusters), porque une un punto a la vez con el vecino más próximo; 'complete' y 'average' producen árboles más balanceados porque penalizan la distancia máxima o promedio entre grupos, no solo el par más cercano.

Ejercicio 3 — Ajuste de hiperparámetros en DBSCAN

- Dataset sintético: dos grupos densos + 10% de outliers distribuidos al azar.

- Se ejecuta DBSCAN con eps en {0.1, 0.3, 0.8} y se cuenta cuántos puntos quedan etiquetados como ruido (-1).
- Conclusión esperada: un eps muy chico sobre-segmenta (incluso puntos válidos caen como ruido); un eps muy grande fusiona los dos grupos densos en uno solo, perdiendo la estructura real.